

系统开发工具基础

第 2 周课后实验报告

姓名：彭俊皓 学号：25020007100

2026 年 9 月 5 日

仓库：<https://github.com/pjh456/sys-devtool/>

本报告覆盖第 2 周课堂四道实验（week2/q05–q08）的操作过程与结果，以及 missing-semester 两讲课后练习：《Shell 入门》第 13–18 题（homework/course-shell/）与《命令行环境》的参数、环境变量、返回码练习（homework/command-line/）；全部结果均在本机重新执行验证。

表 1: 第 2 周课后实验完成情况

部分	内容	关键工具	状态
一	课堂实验回顾（q05–q08）	trap、LSP、pdb、cProfile	🔵 完成
二	课后：《Shell 入门》第 13–18 题	find、xargs、curl、jq	🟢 完成
三	课后：《命令行环境》练习	--、diff、export、返回码	🟠 完成

一

课堂实验回顾（q05–q08）

q05

控制一个可清理的后台任务

📖 题目要求

- 将给定脚本（trap 捕获 TERM/INT 并写 cleanup.log）保存为 q05/worker.sh，赋予执行权限，前台启动并把 stdout、stderr 分别写入 stdout.log、stderr.log；
- 使用 Ctrl-Z 挂起任务，再用 bg 让它在后台继续，用 jobs 确认状态；
- 使用 jobs -p 或等价方式取得 PID（不得手工抄写 PID），发送 SIGTERM 并等待任务结束；
- 确认 cleanup.log 中出现 CLEAN_EXIT 且任务已退出；禁止使用 kill -9。

>_ 操作过程

```
1 chmod +x worker.sh
2 ./worker.sh > stdout.log 2> stderr.log # 前台启动，输出分流到两个日志
```

```
3 ^Z                                # Ctrl-Z 挂起 (SIGTSTP)
4 bg                                # 任务转后台继续运行
5 jobs                              # 确认: [1]+ 运行中 ./worker.sh
6 kill -TERM $(jobs -p)             # 用 jobs -p 取 PID 发 SIGTERM, 不手抄
7 wait                              # 等待任务退出
8 cat cleanup.log                   # 验证清理分支执行
```

✓ 结果与验证

```
$ head -3 stdout.log                # 每秒自增的计数, 共 77 行 (0..76)
0
1
2
$ tail -2 stdout.log
75
76
$ cat stderr.log
已终止                               sleep 1
$ cat cleanup.log
CLEAN_EXIT
```

CLEAN_EXIT 出现在 cleanup.log, 说明 SIGTERM 触发了 trap 的清理命令后才 exit 0; 此时 jobs 中已无该任务。

💡 要点与注意事项

Ctrl-Z 发的是 SIGTSTP (终端停止), bg 让挂起的任务在后台继续; trap 注册的清理命令在收到 TERM/INT 时先执行、再退出; 题目禁用 kill -9 (SIGKILL), 因为 SIGKILL 无法被进程捕获, 清理代码没有执行机会。

q06 语义重构与本地开发反馈

📖 题目要求

按给定代码在 q06 创建三个 Python 文件 (total_price 函数 + 调用 + 测试); 在配置好 Python、pytest、ruff 的环境中启用 Python 语言服务器, 演示跳转定义与查找引用; 用“重命名符号”把 total_price 改为 calculate_total (定义与所有引用同步); 在 app.py 临时加入未使用的 import os, 用 ruff 发现并删除; 最后 ruff 与 pytest 均通过。

🔗 操作过程

给定三文件(math_utils.py 定义 total_price, app.py 调用并打印, test_math_utils.py 断言 12.5 * 4 == 50.0) 创建后:

- 在 app.py 的 total_price 调用处跳转定义, 落点为 math_utils.py 第 1 行; 在定义处查找引用, 命中 app.py 与 test_math_utils.py 各一处;

- 对定义执行 Rename Symbol: `total_price` → `calculate_total`, 语言服务器按引用关系同步修改全部 3 处;
- 在 `app.py` 顶部临时加入 `import os`, 运行 `ruff` 报告后删除该行, 复验。

```
1 .venv/bin/ruff check .          # 含 import os 时报错, 删除后通过
2 .venv/bin/pytest
3 .venv/bin/python app.py
```

✓ 结果与验证

```
$ .venv/bin/ruff check .          # import os 存在时
F401 [*] `os` imported but unused
--> app.py:1:8
$ .venv/bin/ruff check .          # 删除 import os 后
All checks passed!
$ .venv/bin/pytest
test_math_utils.py::test_total_price PASSED
1 passed in 0.01s
$ .venv/bin/python app.py
50.0
```

💡 要点与注意事项

重命名用语言服务器的 Rename Symbol 而非全局文本替换: 它按符号引用关系修改, 不会误伤字符串或注释里的同名内容; `ruff` 的 F401 规则专门检查未使用的导入, 与 `pytest` 一起构成“静态检查 + 动态验证”的本地反馈闭环。

q07 用调试器定位归并排序缺陷

📖 题目要求

给定代码的 `merge` 在 `else` 分支存在缺陷 (`result.append(right[i])`): 先运行给定输入确认结果错误; 用调试器在 `merge` 设断点并观察 `i`、`j`、`left[i]`、`right[j]`; 完成最小修复 (不得用 `sorted` 替换归并排序); 用 `pytest` 增加两个测试: 给定输入与含重复元素的列表。

⚡ 确认缺陷

```
$ python3 merge_sort.py          # 修复前, 给定输入 [3,1,4,1,5,9,2,6]
[1, 5, 4, 3, 4, 5, 6, 9]        # 错误, 应为 [1, 1, 2, 3, 4, 5, 6, 9]
```

🔍 调试定位与最小修复

用 `pdb` 在 `merge` 循环内设断点、单步执行: 第一层调用 `merge([1,3],[1,4])` 走到 `i=1, j=0`

时, `left[i]=3, right[j]=1, 3<=1` 不成立进入 `else` 分支, 代码取的是 `right[i]=right[1]=4`, 而正确值应为 `right[j]=right[0]=1`——缺陷确认。最小修复(1 个字符, `q07/merge_sort.py`):

```
1 -         result.append(right[i]); j += 1    # 此处存在缺陷
2 +         result.append(right[j]); j += 1
```

✓ 结果与验证

按题目要求增加的两个测试 (`test_merge_sort.py`):

- `test_constant_input_array_sort`: 给定输入 `[3, 1, 4, 1, 5, 9, 2, 6]` → `[1, 1, 2, 3, 4, 5, 6, 9]`;
- `test_multiple_element_sort`: 含重复元素 `[3, 4, 5, 5, 1, 2, 3, 7, 8]` → `[1, 2, 3, 3, 4, 5, 5, 7, 8]`。

```
$ .venv/bin/pytest -v
test_merge_sort.py::test_constant_input_array_sort PASSED
test_merge_sort.py::test_multiple_element_sort PASSED
2 passed in 0.01s
```

💡 要点与注意事项

缺陷根因: `else` 分支消费的是 `right` 侧的元素, 索引却误用左指针 `i`; 当 `i==j` 时恰同值不暴露, `i>j` 时必然出错。两个测试正好覆盖“给定输入”与“重复元素”两种要求; 用 `sorted` 会掩盖归并逻辑本身, 题目禁止。

q08 先测量, 再优化慢速词频程序

📖 题目要求

在 q08 用 `generate_words.py` (固定种子 20260831、1000 词表、30000 词) 生成可重复词表; 对故意低效的去重程序用 `time` 运行 2 次记录耗时; 用 `cProfile -s cumulative` 运行一次找出主要耗时位置; 在不改变最终 `count` 的前提下优化 (不得写死 1000); 相同输入再运行 2 次, 计算优化前后中位数与加速比。

🔗 原程序与剖析

```
# wordfreq.py 原版: list 成员判断为 O(n^2)
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
```

```
$ time python3 wordfreq.py          # 两次
real  0m0.121s                    # real  0m0.122s
$ python3 -m cProfile -s cumulative wordfreq.py
1012 function calls in 0.109 seconds
   1    0.108    0.108    0.109    0.109 wordfreq.py:1(<module>)
      # 耗时集中在顶层 for 循环的 `in` 判断 (list 查找  $O(n^2)$ )
```

⚡ 优化

```
# 优化版: dict.fromkeys 一次遍历去重且保序, count 不变、不写死 1000
words = open("words.txt", encoding="utf-8").read().split()
unique = list(dict().fromkeys(words))
print("count=", len(unique))
```

📌 结果与验证

```
$ time python3 wordfreq.py          # 优化后两次
real  0m0.013s                    # real  0m0.013s
$ python3 -m cProfile -s cumulative wordfreq.py
13 function calls in 0.002 seconds
   1    0.001    0.001    0.001    0.001 {built-in method fromkeys}
$ count= 1000                      # 优化前后 count 一致
$ ./calc_speed.sh                  # 中位数与加速比
before version median: 0.121 s
after version median: 0.013 s
Speed up times: 9.35
```

核算: 原程序中位数 $(0.121 + 0.122)/2 = 0.1215\text{s}$, 优化后 $(0.013 + 0.013)/2 = 0.013\text{s}$, $0.1215/0.013 \approx 9.35\times$, 与 `calc_speed.sh` 输出一致。

💡 要点与注意事项

先测量再优化: `cProfile -s cumulative` 把嫌疑直接指到模块顶层的 `for` 循环 (`wordfreq.py:1` 占 $0.108/0.109\text{s}$), 而非凭感觉改; `dict.fromkeys` 在 C 层一次完成“去重 + 保序”, 把 $O(n^2)$ 降到 $O(n)$, 语义与原程序完全等价。

二 课后练习: missing-semester 《Shell 入门》第 13–18 题

第 13 题 home 目录最常见的 5 种文件扩展名

📄 题目要求

使用管道找出 home 目录中最常见的 5 种文件扩展名。（提示：组合 find、grep/sed/awk、sort、uniq -c 以及 head）

🔗 解答 (course-shell/13.sh)

```
1 find . ~ -type f -exec basename {} \; \  
2 | sed "s/.*\./" \  
3 | sort --reverse \  
4 | uniq -c | sort --numeric-sort --reverse \  
5 | head -5
```

✅ 结果与验证

```
$ sh 13.sh  
121086 js  
62115 html  
49495 o  
47974 py  
36275 h
```

输出为扩展名计数的实测结果（home 下 node 模块的 .js、浏览器缓存的 .html 等构成前三）。复跑验证用 fd（约 3s）：

```
fd -H --no-ignore -t f . . ~ | sed 's|.*\.|'| \  
| sort --reverse | uniq -c | sort --numeric-sort --reverse | head -5
```

与 13.sh 的输出完全一致。

💡 要点与注意事项

sed "s/.*\./" 取最后一个点之后的部分即扩展名 (a.b.c → c); uniq -c 只统计相邻行，所以计数前必须先 sort；末段 sort -nr | head -5 按次数取前 5。-exec basename {} 对每个文件 fork 一个进程，在本机 home 规模下超过 2 分钟跑不完，故复跑改用 fd：-H 包含隐藏文件、--no-ignore 不理睬 .gitignore 等忽略规则、-t f 只取普通文件，与 find -type f 的文件集一致，计数因此完全相同。

第 14 题 find + xargs 统计.sh 行数

📄 题目要求

xargs 会把 stdin 的每一行转换为命令参数。结合 find 和 xargs（不要用 find -exec）找出目录中所有 .sh 文件并用 wc -l 统计每个文件行数；加分项：正确处理文件名中的空格（提示：-print0 和 -0）。

</> 解答 (course-shell/14.sh)

```
1 if [ $# -lt 1 ]; then
2     echo "Directory cannot be empty!"
3     exit 1
4 fi
5 DIR=$1
6 if [ ! -d $DIR ]; then
7     echo "Given path is not a directory!"
8     exit 1
9 fi
10 find $DIR -type f -name "*.sh" -print0 | xargs -0 wc -l
```

✓ 结果与验证

```
$ sh 14.sh /home/pjh123/homework/sys-devtool/homework
1 /home/pjh123/.../homework/course-shell/3/lsfile.sh
1 /home/pjh123/.../homework/course-shell/3/lsstar.sh
2 /home/pjh123/.../homework/course-shell/15.sh
152 总计
```

逐文件行数 + 未行总计；对含空格的目录参数做了非空与目录存在性校验。

💡 要点与注意事项

-print0 以 \0 而非换行分隔路径，xargs -0 按 \0 切分参数，二者配对后文件名中的空格、换行都不会导致分词错误；不用 -exec 也满足题目“xargs 转换参数”的要求。

第 15 题 curl 统计课程讲数

📖 题目要求

使用 curl 获取课程网站 <https://missing.csail.mit.edu/> 的 HTML，通过 grep 统计列出了多少讲。（提示：找出每讲课程名称在 HTML 中的共性；用 curl -s 关闭进度输出）

</> 解答 (course-shell/15.sh)

```
1 curl -s https://missing.csail.mit.edu/ | grep "href=\"/20\" | wc -l
```

✓ 结果与验证

```
$ sh 15.sh
20
```

课程主页 2026 年栏目里每讲链接形如 href="/2026/...", 以 href="/20 为共性计数得 20 个讲次链接。

💡 要点与注意事项

`curl -s` 关闭进度条，避免把进度信息混进管道；用“每讲链接的公共前缀”而非具体讲名做 `grep`，课程增减讲时脚本依然成立。

第 16 题 `curl + jq` 处理 JSON

📖 题目要求

用 `curl` 获取示例数据 `https://microsoftedge.github.io/Demos/json-dummy-data/64KB.json` 再用 `jq` 提取 `version` 大于 6 的人员姓名。（提示：先 `jq .` 看结构；再试 `jq '.[[] | select(...)|.name'`）

🔗 解答 (`course-shell/16.sh`)

```
1 curl https://microsoftedge.github.io/Demos/json-dummy-data/64KB.json \  
2 | jq '.[[] | select(.version > 6) | .name'
```

✅ 结果与验证

```
$ sh 16.sh | head -4  
"Adeel Solangi"  
"Aamir Solangi"  
"Adil Eli"  
"Adil Abro"  
$ sh 16.sh | wc -l  
89
```

💡 要点与注意事项

`jq` 过滤链：`.[[]` 展开数组元素 → `select(.version > 6)` 保留满足条件的对象 → `.name` 投影出姓名字段；先用 `jq .` 观察顶层结构（人员数组）再写选择器，避免猜字段名。

第 17 题 `awk` 按列过滤并交换列

📖 题目要求

`awk` 可以按列值过滤行并改写输出。写一个 `awk` 命令：只输出第二列大于 100 的行，并交换第一列和第三列。测试命令：`printf 'a 50 x\nb 150 y\nc 200 z\n'`

🔗 解答 (`course-shell/17.sh`)

```
1 printf 'a 50 x\nb 150 y\nc 200 z\n' | awk '$2 > 100 { print $3, $2, $1 }'
```


✓ 结果与验证

```
$ sh 17.sh  
y 150 b  
z 200 c
```

第一行 (50) 被过滤；保留行的第一、三列互换，第二列位置不变。

💡 要点与注意事项

条件表达式放在模式位置 ($\$2 > 100$)，动作块里 `print $3, $2, $1` 直接以新顺序输出三列，无需先存中间变量； $\$2$ 默认按字符串/数值双态比较，纯数字列可安全做数值判断。

第 18 题 拆解 SSH 日志管道 + history 最常用命令

📖 题目要求

拆解讲义中的 SSH 日志处理管道：每一步分别做了什么？然后仿照它构建一个管道，从 `~/.bash_history`（或 `~/.zsh_history`）中找出最常用的 Shell 命令。

⚡ 管道拆解 (course-shell/18/explain.txt)

原管道共 7 步，逐步拆解（摘要自 explain.txt）：

1. `ssh myserver '...'`：登录远端执行引号内命令；`journalctl -u sshd -b-1` 取 `sshd` 服务上次启动以来的日志；`grep "Disconnected from"` 筛出断开连接行；
2. `sed -E 's/...\1/'`：正则替换，`\1` 取 `(.*)` 捕获的用户名，`[`]+ port.*` 吃掉行尾无关内容；
3. `sort | uniq -c`：排序后统计每个用户名的出现次数；
4. `sort -nk1,1`：按次数字段（第 1 列）数值升序；
5. `tail -n10`：保留末尾 10 行，即次数最多的 10 个用户；
6. `awk 'print $2'`：去掉次数列，只留用户名；
7. `paste -sd,`：全部合并为一行、以逗号分隔。

最终输出即“上次启动以来 ssh 断联次数最多的 10 个用户”。

➤ 仿写：最常用命令 (course-shell/18/check_command.sh)

```
1 cat ~/.bash_history | sort | uniq -c | sort -nk1,1 | tail -n1 \  
2 | awk '{ print $2 }'
```

✓ 结果与验证

```
$ sh 18/check_command.sh  
clear
```

与讲义同构：history 流 → sort | uniq -c 计数 → sort -nk1,1 | tail -n1 取最高频 → awk 投影出命令本身；本机 history 中最高频的是 clear。

💡 要点与注意事项

仿写时把 tail -n10 收紧为 tail -n1 即得“最常用的一条”；awk 'print \$2' 只取每行第 2 个字段（命令名），不含参数；若 history 跨行存储（多行命令），此法会按行计数，属于该简化的已知局限。

三 课后练习：missing-semester 《命令行环境》

参数·1 -- 选项终止符

📖 题目要求

cmd --flag -- --notafalg 中的 -- 是特殊参数：告诉程序其后不再解析选项，全部按位置参数处理。这有什么用？运行 touch -- -myfile，然后在不使用 -- 的情况下把它删掉。

🔗 解答 (command-line/params/1.sh)

```
1 touch -- -myfile  
2 rm ./-myfile
```

✓ 结果与验证

```
$ sh 1.sh  
$ ls -la -- -myfile      # 确认已删除  
ls: 无法访问 '-myfile': 没有那个文件或目录
```

💡 要点与注意事项

没有 -- 时，-myfile 会被 touch 当作选项解析（未知选项直接报错）；删除时改用 ./-myfile：以 ./ 开头的路径不再是 - 起始，无需 -- 也不会被误认为选项。

参数·2 ls 组合选项

📖 题目要求

阅读 man ls，写出一条 ls 命令：包含所有文件（含隐藏文件）、文件大小以易读格式显示、按最近修改时间排序、输出带颜色。

🔗 解答 (command-line/params/2.sh)

```
1 ls -alt --human-readable --color
```

✅ 结果与验证

在 week2/q08 下运行（终端中目录名带颜色，此处略去）：

```
$ sh 2.sh | head -4
总计 200K
drwxr-xr-x 4 pjh123 pjh123 4.0K  8月31日 14:28 .
-rw-r--r-- 1 pjh123 pjh123   43  8月31日 14:28 .gitignore
-rw-r--r-- 1 pjh123 pjh123  194  8月31日 14:27 generate_words.py
```

💡 要点与注意事项

四个要求对应 -a（含隐藏）、-h（长选项 --human-readable，43 → 4.0K 量级可读）、-t（按 mtime 排序）与 --color；单横线短选项可合并为 -alt，且选项顺序无关（-alt 与 -tal 等价）。

参数·3 printenv 与 export 的差异

📖 题目要求

进程替换 <(command) 可以把命令输出当成文件用。配合 diff 和进程替换比较 printenv 与 export 的输出，解释它们为什么不一样。（提示：diff <(printenv | sort) <(export | sort)）

>_ 操作

```
1 diff <(printenv | sort) <(export | sort) | head -6
2 export | head -2
```

✅ 结果与验证

```
$ diff <(printenv | sort) <(export | sort) | head -3
1,58c1,58
< AGENT=1
```

```
< CARGO_BUILD_JOBS=8
$ diff <(printenv | sort) <(export | sort) | sed -n '60,63p'
---
> declare -x AGENT="1"
> declare -x CARGO_BUILD_JOBS="8"
$ export | head -2
declare -x AGENT="1"
declare -x CARGO_BUILD_JOBS="8"
```

两边共 58 个变量逐行不同 (1,58c1,58), 差异来源 (params/3.txt):

- **格式不同:** printenv 输出 key=value; export 输出 declare -x key="value";
- **空值处理不同:** printenv 不显示空值的导出变量, export 会显示; 可用 declare +x KEY 取消导出。

💡 要点与注意事项

<(cmd) 是进程替换: shell 把命令输出写入临时文件 (如 /dev/fd/63), 再用该文件名替换 <(…), 于是 diff 可以把两个“只存在于管道中的流”当普通文件比较, 无需先落盘成临时文件。

环境变量·1 marco / polo

📖 题目要求

写两个 bash 函数 marco 与 polo: 执行 marco 时保存当前工作目录; 之后无论切到哪个目录, 执行 polo 都应 cd 回 marco 时的目录。代码写进 marco.sh, 通过 source marco.sh 加载到当前 shell。

🔗 解答 (command-line/env/marco.sh)

```
1 marco() {
2     export MARCO_POLO_DIR="$(pwd)"
3 }
4 polo() {
5     if [ -z "$MARCO_POLO_DIR" ]; then
6         echo "Error: Run 'marco' first!"
7         return 1
8     fi
9     cd "$MARCO_POLO_DIR" || echo "Error: Directory no longer exists!"
10 }
```

✓ 结果与验证

```
$ source marco.sh
$ polo                                # 尚未执行 marco
Error: Run 'marco' first!
$ marco
$ cd /
$ polo
$ pwd
/tmp/opencode                        # 回到 marco 时的目录
```

未先 marco 时 polo 报错并返回 1；正常路径下 marco → cd / → polo 准确返回原目录。

💡 要点与注意事项

判空必须写 `[ -z "$MARCO_POLO_DIR" ]`（展开变量）：若写成 `[ -z "MARCO_POLO_DIR" ]`，检查的是字符串字面量，永远非空，错误分支不会触发；`export` 使变量进入环境变量，子进程也能继承；函数必须用 `source` 加载——`./marco.sh` 会在子 shell 中执行，子 shell 退出后函数定义随之消失。

返回码·1 循环运行直到失败

📖 题目要求

假设你有一个很少失败的命令。写一个 bash 脚本：不断运行给定的 worker 脚本直到它失败，把标准输出和标准错误分别保存到文件，最后把结果打印出来；如果还能顺便报告它运行了多少次才失败，就加分。（worker 脚本以 `RANDOM % 100 == 42` 为失败条件，失败时 `exit 1` 并向 `stderr` 写错误信息）

🔗 解答 (command-line/retcode/)

```
1 # runner.sh
2 CNT=0
3 while true; do
4     if ./worker.sh 1> stdout.log 2> stderr.log; then
5         ((CNT++))
6     else
7         cat stdout.log
8         cat stderr.log
9         echo "Error happens after $CNT trials"
10        break
11    fi
12 done
```

✓ 结果与验证

```
$ ./runner.sh
Something went wrong
The error was using magic numbers
Error happens after 41 trials
$ cat stdout.log
Something went wrong
$ cat stderr.log
The error was using magic numbers
```

成功轮次的输出被 `stdout.log` 覆盖、终端保持安静；失败轮次才把两条流分别落盘并原样打印，CNT 报告成功轮数（加分项）。

💡 要点与注意事项

worker 以 `exit 1` 表示失败，`if` 依据返回码分支（0 = 成功，非 0 = 失败）——这正是 Shell 的“返回码即接口”约定；`1>` 与 `2>` 把两条流分开保存，失败后可分别核对“程序说什么”与“错误是什么”；失败概率 1/100，平均约 100 次命中，本次运行成功 41 轮、第 42 次失败，与预期量级一致。

四 总结与收获

• 课堂实验（q05–q08）

- Ctrl-Z 对应 SIGTSTP 挂起、bg/fg 切换前后台；trap 让脚本在 TERM/INT 上先清理再退出，SIGKILL 无法被捕获所以题目禁用 `kill -9`。
- 语言服务器的 Rename Symbol 按引用关系同步重命名，比全局文本替换安全；ruff 查静态问题、pytest 跑动态验证。
- 调试器定位缺陷的关键是单步到“实际值与期望值分叉”的那一轮，对照 i、j 与两侧元素即可看出索引取错。
- 性能工作流是“先测量、再优化、再复测”：cProfile -s cumulative 找热点，time 各测多次取中位数，最后算加速比。

• 课后练习（Shell 13–18 + 命令行环境）

- `uniq -c` 只统计相邻行，计数前必须 sort；取 Top-N 用 `sort -nr | head`。
- `-print0 | xargs -0` 是处理含空格文件名的标准搭配。
- jq 的“展开 → select → 投影”三步过滤链能替代大段手工解析。
- `--` 终止选项解析、`./` 前缀改写路径，二者都是应对 `-` 开头文件名的手段。
- 进程替换 `<(cmd)` 让“流”以文件名身份参与命令，比较两个命令输出不再需要临时文件。

- 环境变量经 `export` 进入子进程环境；shell 函数必须 `source` 进当前 shell 才存活。
- 返回码驱动 `if/while`，“循环运行直到失败 + 分流通路落盘 + 计数”是复现偶发故障的通用骨架。